Ricerca Operativa

Contents

1 Introduzione **4**

2 Processo decisionale **5**

3 Modelli di ottimizzazione **6**
3.1 Programmazione lineare e lineare intera 6
3.2 Pianificazione della produzione (esempio) 6
3.2.1 Descrizione del problema 6
3.2.2 Inizializzazione 7
3.2.3 Traduzione in grafico e soluzione 7
3.3 Possibili soluzioni dei problemi di ottimizzazione 7

4 Tecnica di modellazione di problemi **8**
4.1 Il problema della fonderia (esempio) 8
4.1.1 Descrizione del problema 8
4.1.2 Inizializzazione 8
4.1.3 Un problema difficile: generalizzazioe 9

5 Variabili binarie **11**
5.1 Problema dello zaino (esempio) 11
5.1.1 Descrizione del problema 11
5.1.2 Inizializzazione 11
5.1.3 Rappresentazione relazioni logiche 11
5.2 Albero di copertura di costo minimo (esempio applicativo) 13
5.2.1 Inizializzazione 13
5.2.2 Problema del commesso viaggiatore (applicato al grafo) 13
5.3 Problema di assegnamento (esempio applicativo) 14
5.3.1 Inizializzazione 14
5.4 Assegnamento di frequenze (esempio applicativo) 14
5.4.1 Inizializzazione 15

6 Partizione di sottoinsiemi **16**
6.1 Problema di partizione 16
6.1.1 Descrizione del problema 16
6.1.2 Inizializzazione 16
6.2 Problema di configurazione 18
6.2.1 Descrizione del problema 18
6.3 Problema di instradamento 20
6.3.1 Descrizione del problema 20
6.3.2 Inizializzazione 20

7 Problemi su reti: problemi di flusso e di cammino **22**
7.1 Problema delle fognature (esempio) 22
7.1.1 Descrizione del problema 22
7.1.2 Inizializzazione 23
7.2 Problema del flusso di costo minimo 24

7.2.1 Descrizione del problema . . . 24
7.2.2 Inizializzazione . . . 24
7.3 Problema del cammino minimo . . . 25
7.3.1 Descrizione del problema . . . 25
7.3.2 Inizializzazione . . . 25

8 Variabili a valori discreti 26
8.1 Problema delle fognature ampliato . . . 26
8.1.1 Descrizione del problema . . . 26
8.1.2 Inizializzazione . . . 26

9 Variabili semicontinue 27

10 Costi di setup 28

11 Massimizzare minimi e minimizzare massimi 29
11.1 Tempi di completamento delle barche (esempio) . . . 29
11.1.1 Descrizione del problema . . . 29
11.1.2 Inizializzazione . . . 29

12 Linearizzazione valore assoluto 30
12.1 Equipartizione (esempio) . . . 30
12.1.1 Descrizione del problema . . . 30
12.1.2 Inizializzazione . . . 30

13 Vincoli disgiuntivi 31
13.1 Grafo spazio temp e probabilità (esempio) . . . 31
13.1.1 Descrizione del problema . . . 31
13.1.2 Modellazione . . . 32

14 Grafi e reti di flusso 33
14.1 Definizioni iniziali . . . 33
14.2 schedulazione della produzione (esempio) . . . 34
14.2.1 Descrizione . . . 34
14.2.2 Inizializzazione . . . 34
14.3 Trasformazioni equivalenti . . . 35
14.3.1 Sorgenti e pozzi multipli . . . 36
14.3.2 Capacità inferiore non nulle . . . 36
14.3.3 Limiti di capacità sui nodi . . . 37
14.3.4 Eccesso di domanda o offerta . . . 37
14.3.5 Ispezioni su una linea di produzione (esempio) . . . 38

15 Cammini di costo minimo 40
15.1 Definizione del problema generico . . . 40
15.2 Particolarità e proprietà . . . 40
15.3 Shortest Path Tree (SPT) . . . 41
15.3.1 Albero ottimo . . . 41
15.3.2 Teorema di ottimalità . . . 41

1 Introduzione

Corso E-Learning: Ricerca Operativa

La **ricerca operativa** è la disciplina che si occupa di modellizzazione e risoluzione di problemi decisionali complessi con metodologie matematiche, dalla statistica all'informatica. Il processo è complesso e non lineare.

Ottimizzazione: ricerca del modo migliore per svolgere un azione, concetto importante della ricerca operativa. Tale concetto nasce durante la Seconda Guerra Mondiale, infatti la ricerca operativa nasce dallo studiare l'ottimizzazione di strategie militari.

Alcuni esempi di applicazioni della ricerca operativa:

- Scheduling negli aereoporti per merce
- Problema dei sette ponti
- Routing dei dati

Nota importante: molti concetti di questo corso sono introdotti gradualmente tramite esempi pratici, pertanto nei relativi capitoli non sarà presente una spiegazione immediata dell'argomento.

2 Processo decisionale

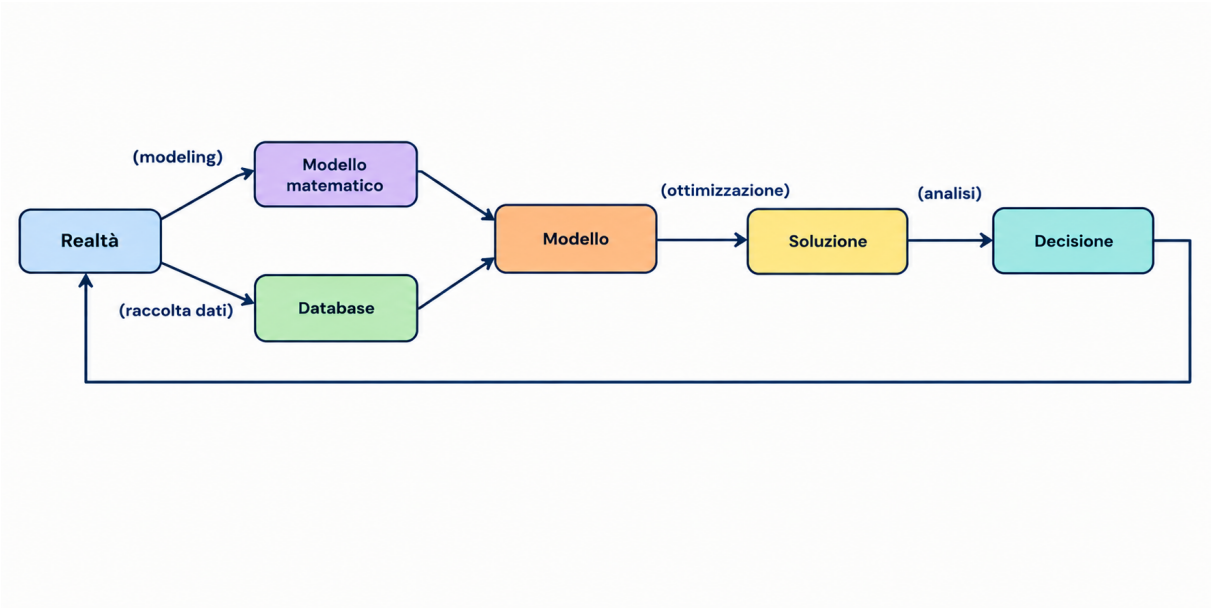


Figure 1: Processo decisionale - diagramma a blocchi.

- **Realtà:** si parte da un problema, una realtà di interesse.
- **Database:** si raccolgono i dati necessari per descrivere il problema e trovare una soluzione.
- **Modello matematico:** si convertono informazioni in variabili e dati matematici, come vincoli e funzione obiettivo.
- **Modello:** i dati e il modello matematico vengono combinati per ottenere i valori effettivi del problema e creare quindi il modello del problema specifico.
- **Soluzione:** trovata attraverso le operazioni di ottimizzazione del modello specifico.
- **Decisione:** una volta analizzata e processata la soluzione si decide come procedere.

Il processo decisionale è ciclico perchè la decisione viene applicata alla realtà, che può presentare nuovi problemi.

3 Modelli di ottimizzazione

Un **modello di ottimizzazione** è una rappresentazione matematica di un problema decisionale.
Un modello tale serve a risolvere un problema di ottimizzazione e per farlo utilizza questi:

- **Funzione Obiettivo:** rappresenta ciò che vogliamo massimizzare o minimizzare (profitto, produzione...)
- **Variabili decisionali:** incognite del modello
- **Vincoli:** rappresentano le limitazioni del problenma, come il budget.
- **Parametri:** valori fissi, influenzano funzione obiettivo e vincoli ma non possono essere modificati.

3.1 Programmazione lineare e lineare intera

I nostri modelli devono essere lineari.

Programmazione lineare: se i vincoli e la funbzione obiettivo sono lineari e le variabili sono continue.

Programmazione lineare intera: se i vincoli e la funzione obiettivo sono lineari e alcune delle variabili sono discrete (o **intere**).

Nei nostri modelli non si può:

- Dividere o moltiplicare per variabili
- Utilizzare valori assoluti

Si possono solo utilizzare somme e moltiplicazioni o divisioni per parametri.

3.2 Pianificazione della produzione (esempio)

Osserviamo adesso un problema d'esempio per apprendere come trovare un modello di ottimizzazione e quindi una soluzione ottimizzata.

3.2.1 Descrizione del problema

La società Pintel deve pianificare la produzione della sua fabbrica di microprocessori. L'azienda possiede due diverse linee di prodotti: processori Pintium, venduti a 500 euro, ed i Coloron, venduti a 200 euro perche meno potenti, destinati ai consumer semplici.
L'impianto è in grado di realizzare 3000 "wafer" alla settimana, su ogni wafer trovano posto 500 coloron oppure 300 Pintium.
La resa di un wafer dipende dal tipo di processore:

- Pintium: 50%
- Coloron: 60%

Ogni settimana si può mettere sul mercato un massimo di 400.000 unità per evitare il calo dei prezzi per i Pintium, 700.000 per i Coloron invece.
Si vuole capire quale quantità di ciascun modello bisonerebbe produrre settimanalmente per massimizzare il profitto.

3.2.2 Inizializzazione

Iniziamo con la preparazione dei nostri dati, vediamo le variabili:

- X_P: numero di processori Pintium.
- X_C: numero di processori Coloron.

La funzione obiettivo è:

$$maxZ = 500X_p + 200X_C$$

I vincoli sono:

$$\begin{cases} 0 \leq X_P \leq 400.000 \\ 0 \leq X_C \leq 700.000 \\ \frac{X_P}{300 \cdot 0,5} + \frac{X_C}{500 \cdot 0,6} \leq 300 \end{cases}$$

3.2.3 Traduzione in grafico e soluzione

Per disegnare il grafico corrispondente inanzitutto è consigliato semplificare i dati comprimendoli (per esempio quantità di centinaia o migliaia).
Bisogna rappresentare i vincoli e ridurre il dominio, trovando lo **spazio soluzione**.
Una volta fatto questo, bisogna massimizzare la funzione obiettivo con l'utilizzo del gradiente, in questo caso:

$$\nabla z = (500, 200)$$

La soluzione consiste nell'ultimo punto in cui il gradiente, scorrendo verso destra, rimane nel dominio.

3.3 Possibili soluzioni dei problemi di ottimizzazione

Un problema di ottimizzazione può avere diversi tipi di soluzione, tra cui:

- **Un'unica soluzione ottima:** singola soluzione perfetta.
- **Soluzioni infinite:** non esiste una singola soluzione ottima.

  $z = 10 \rightarrow$ tutto lo spazio soluzione.

  Punti infiniti, soluzioni infinite.

- **Problema illimitato:** spazio soluzione non chiuso (superiormente o inferiormente), non esiste una soluzione, bensì è ∞.
- **Nessuna soluzione:** lo spazio soluzione è vuoto.

4 Tecnica di modellazione di problemi

Alcuni problemi possono essere molto complessi, è quindi necessario sapere come modellarli per renderli risolvibili.

4.1 Il problema della fonderia (esempio)

4.1.1 Descrizione del problema

Una fonderia deve produrre 1000 pezzi del peso ciascuno di 1 kg. Il ferro con cui tali pezzi sono fatti dovrà contenere almeno il $3,25\%$ e al più $5,5\%$ di silicio, inoltre dovrà contenere al meno lo $0,45\%$ di manganese. Per il mix, sono disponibili tre tipi di materiale ferroso con le caratteristiche in foto.
Inoltre, si può aggiungere direttamente manganese al costo di $10/kg$.

Si vuole determinare il piano di produzione che minimizza il costo del materiale utilizzato.

Materiale ferroso	1	2	3
Silicio (%)	4,00	1,00	0,60
Manganese (%)	0,45	0,50	0,40
Costo (£/kg)	0,025	0,030	0,018

Table 1: Composizione e costo dei materiali

4.1.2 Inizializzazione

Iniziamo col definire delle costanti:

- $n = 4$, numero dei materiali.
- $K = 1000$, numero di pezzi da produrre.
- e_i, costo del materiale i.
- s_i, percentuale di silicio del materiale i.
- m_i, percentuale di manganese del materiale i.
- $s, \overline{S}$, percentuale minima e massima di silicio.
- m, percentuale minima di manganese.

Iniziamo con le variabili:

- X_1: materiale 1
- X_2: materiale 2
- X_3: materiale 3
- X_4: manganese extra

Poi i vincoli:

$$\begin{cases} X_1, X_2, X_3, X_4 \geq 0 \\ 3,25 \cdot 1000 \leq 4 \cdot X_1 + X_2 + 0,6 \cdot X_3 \leq 5,5 \cdot 1000 \\ 0,45 \cdot X_1 + 0,5 \cdot X_2 + 0,4 \cdot X_3 + 100 \cdot X_4 \geq 0,45 \cdot 1000 \end{cases}$$

La funzione obiettivo invece è:

$$minZ = 0,25 \cdot X_1 + 0,03 \cdot X_2 + 0,018 \cdot X_3 + 10 \cdot X_4$$

Adesso la fonderia vuole aggiungere un nuovo vincolo al mix di fusione: adesso il materiale 1 rispetto ai materiali ferrosi utilizzati nella fusione deve rappresentare almeno il 20%. Scriviamolo subito e aggiorniamo i vincoli:

$$\begin{cases} X_1, X_2, X_3, X_4 \geq 0 \\ 3,25 \cdot 1000 \leq 4 \cdot X_1 + X_2 + 0,6 \cdot X_3 \leq 5,5 \cdot 1000 \\ 0,45 \cdot X_1 + 0,5 \cdot X_2 + 0,4 \cdot X_3 + 100 \cdot X_4 \geq 0,45 \cdot 1000 \\ \frac{X_1}{X_1+X_2+X_3} \geq 0,20 \end{cases}$$

Questo non va bene perchè non è lineare, spostiamo il denominatore a destra:

$$\begin{cases} X_1, X_2, X_3, X_4 \geq 0 \\ 3,25 \cdot 1000 \leq 4 \cdot X_1 + X_2 + 0,6 \cdot X_3 \leq 5,5 \cdot 1000 \\ 0,45 \cdot X_1 + 0,5 \cdot X_2 + 0,4 \cdot X_3 + 100 \cdot X_4 \geq 0,45 \cdot 1000 \\ X_1 \geq 0,20 \cdot (X_1 + X_2 + X_3) \end{cases}$$

4.1.3 Un problema difficile: generalizzazioe

Il problema è molto complesso ed è necessario semplificarlo / generalizzarlo.
Ecco come si aggiornano i vincoli per renderli generali:

$$min = \sum_{i=1}^{n} c_i \cdot X_i$$

$$min = \sum_{i=1}^{n} X_i = K$$

$$s \cdot K \leq min = \sum_{i=1}^{n-1} s_i \cdot X_i \leq \overline{S} \cdot K$$

$$\sum_{i=1}^{n} m_i \cdot X_i \geq m \cdot K$$

$$x_i \geq 0, \forall i = \{1, \ldots, n\}$$

Spiegando riga per riga:

1. La funzione obiettivo corrisponde alla somma dei costi di ogni pezzo prodotto.
2. La somma di tutti i pezzi prodotti corrisponde alla quantità di pezzi da produrre.

3. La quantità di silicio totale in ogni pezzo deve rispettare i limiti massimi e minimi specificati prima. L'ultimo materiale è escluso perchè è puro manganese.

4. La quantità di manganese totale in ogni pezzo prodotto deve rispettare la soglia minima specificata prima.

5. La variabile che conta i pezzi per un qualsiasi materiale è non negativa.

5 Variabili binarie

5.1 Problema dello zaino (esempio)

5.1.1 Descrizione del problema

Un investitore dispone di un capitale limitato B da destinare a una serie di $n \in N$ progetti di investimento.
Ogni progetto è caratterizzato da:

- un **costo di investimento** c_i, per ogni progetti $i \in I = \{1, \ldots, n\}$.
- un **rendimento atteso** w_i, per ogni progetto $i \in I = \{1, \ldots, n\}$.

L'investitore deve decidere quali progetti finanziare per massimizzare il rendimento atteso complessivo dato il capitale disponibile.
I parametri sono tutti non negativi (dato che è specificato nella descrizione, non deve essere un vincolo, ma un controllo preliminare).

5.1.2 Inizializzazione

Partiamo con le costanti:

- B: capitale.
- n: progetti.
- c_i: costo del progetto n_i con $i \in I = \{1, \ldots, n\}$
- w_i: rendimento del progetto n_i con $i \in I = \{1, \ldots, n\}$.

Ora le variabili:

- X_i: variabile binaria, **(1)** se prendo il progetto, **(0)** altrimenti.

Ora la funzione obiettivo:

$$maxZ = \sum_{i \in I} w_i \cdot X_i$$

In questo modo la funzione conta solo i progetti scelti per massimizzare la soluzione.
Adesso guardiamo i vincoli:

$$\left\{ \sum_{i \in I} c_i \cdot X_i X_1 = \{0, 1\} \right.$$

5.1.3 Rappresentazione relazioni logiche

Si aggiungono nuovi vincoli:

1. Ci sono due progetti in competizione (A,B). L'investitore intende finanziare esattamente uno tra essi.
2. E se volesse finanziarne al minimo uno?
3. E se volesse finanziarne al massimo uno?

Ecco come si rappresentano:

1. $X_A + X_B = 1$
2. $X_A + X_B \geq 1$
3. $X_A + X_B \leq 1$

Adesso, consideriamo che se si investe in A, allora si investe in B, ma se non si investe in A non è necessario investire in C.
Ecco come si rappresenta:

$$X_A \leq X_B$$

Infine:

- Se finanziamo A e B, allora finanziamo anche C.
- Si può finanziare il progetto C solo se si finanziano A e B.

Si ha una doppia implicazione:

$$\begin{cases} X_C \geq X_A + X_B - 1 \\ X_C \leq \frac{X_A + X_B}{2} \end{cases}$$

Vediamo adesso altri vincoli:

- Si deve finanziare il progetto C se si finanzia almeno uno tra A o B
- Si può finanziare il progetto C se si finanzia almeno uno tra A o B

Qui c'è una ridondanza, si considera solo la prima condizione dato che è più **rilevante**, vediamo comunque entrambi i vincoli:

$$\begin{cases} X_C \geq \frac{X_A + X_B}{2} \\ X_C \leq X_A + X_B \end{cases}$$

Vediamo adesso ancora altri vincoli:

- Si deve finanziare il progetto C se si finanzia esattamente uno tra A o B
- Si può finanziare il progetto C se si finanzia esattamente uno tra A o B

Ecco i vincoli:

$$\begin{cases} X_C \geq X_A - X_B \\ X_C \geq X_B - X_A \end{cases}$$

$$\begin{cases} X_C \leq 2 - X_A - X_B \\ X_C \leq X_A + X_B \end{cases}$$

Nel caso appena svolto si sarebbe potuto solitamente usare un valore assoluto, ma non in ricerca operativa.

5.2 Albero di copertura di costo minimo (esempio applicativo)

Vediamo un'applicazione interessante delle variabili binarie.
Dato un grafo connesso:

$$G = (V, E)$$

Dove i vertici rappresentano delle città e gli archi delle possibili connessioni tra esse.
Ciascun arco ha un costo:

$$c_{ij} > 0$$

Vogliamo fare in modo che ogni città sia raggiungibile da ogni altra città attraverso la rete, con il minor costo.

5.2.1 Inizializzazione

Variabili:

$$X_{ij} = \begin{cases} 1, \text{lato in percorso soluzione} \\ 0, altrimento \end{cases}$$

Funzione obiettivo:

$$minZ = \sum_{ij \in E} c_{ij} \cdot X_{ij}$$

Vincoli:

$$\begin{cases} X_{ij} = \{0, 1\} \\ \{ij\} \in E | i \in S, j \in V \setminus S, \sum X_{ij} \geq 1, \forall S, V; S \neq \emptyset \end{cases}$$

Il secondo vincolo garantisce che il grafo sia **connesso**.
Non procederemo alla realizzazione del modello di questo problema dato che è servito solo per dimostrare come usare le variabili logiche in questo contesto.

5.2.2 Problema del commesso viaggiatore (applicato al grafo)

Vediamo adesso un'altra situazione: il commesso viaggiatore deve effettuare un **ciclo hamiltoniano di costo minimo**, ovvero un ciclo che visita una volta sola ogni città, torna alla città di partena e ha un costo minimo.
Prepariamo tutto, prima le variabili:

$$X_{ij} = \begin{cases} 1, \{ij\} \in E \text{ e in soluzione} \\ 0 \end{cases}$$

Funzione obiettivo:

$$minZ = \sum_{\{ij\} \in E} C_{ij} \cdot X_{ij}$$

Vincoli:

$$\begin{cases} \sum_{ij \in E} X_{ij} = |V| \\ \{ij\} \in E | i \in S, j \in V \setminus S, \sum X_{ij} \geq 1, \forall S, V; S \neq \emptyset \\ \sum_{ij \in E} X_{ij} = 2 \end{cases}$$

In realtà non è necessario che siano precisamente 2 gli archi di ogni nodo, bensì devono essere semplicemente un numero pari.

5.3 Problema di assegnamento (esempio applicativo)

Un'azienda deve assegnare alcune attività da svolgere ad un insieme di lavoratori. I lavoratori e le attività sono due insiemi:

- L = lavoratori
- A = attività

Supponiamo che $i \in L$ e $j \in A$.
Ogni attività ha un costo:

$$c_{ij}$$

Si vogliono assegnare tutte le attività ai lavoratori e che ogni lavoratore abbia una ed una sola attività assegnata, minimizzando il costo.

5.3.1 Inizializzazione

Dato che c'è una biiezione sappiamo che:

$$|L| = |A|$$

Sviluppiamo un grafo:

$$G = (L \cup A, E)$$

Variabili:

$$X_{ij} = \begin{cases} 1, \text{se assegno attività j al lavoratore i} \\ 0 \end{cases}$$

Funzione obiettivo:

$$minZ = \sum_{E} c_{ij} \cdot X_{ij}$$

Vincoli:

$$\begin{cases} \forall j \in A, \sum_{i \in L} X_{ij} = 1 \\ \forall i \in L, \sum_{j \in A} X_{ij} = 1 \\ X_{ij} \in \{ij\} \end{cases}$$

5.4 Assegnamento di frequenze (esempio applicativo)

Una compagnia di telecomunicazioni deve decidere come coprire la cottà attraverso un insieme di stazioni radio.
Ogni città ha una stazione radio, ma le città vicine devono avere frequenze diverse tra esse per non avere interferenze.
Ogni frequenza ha un costo:

$$c_{if}$$

con:

$$i \in S = \text{citta / antenne}$$

$$f \in F = frequenze$$

Si vuole coprire la città con il numero minore di frequenze.

5.4.1 Inizializzazione

Variabili:

$$y_f = \begin{cases} 1, \text{se uso } f \in F \\ 0 \end{cases}$$

$$X_{if} = \begin{cases} 1, \text{se associo f a i} \\ 0 \end{cases}$$

Funzione obiettivo:

$$minZ = \sum_{f \in F} y_f$$

Vincoli:

$$\begin{cases} \sum_{f \in F} X_{if} = 1, \forall i \in S \\ X_{if} \in \{0, 1\} \\ y_f \in \{0, 1\} \\ X_{if} + X_{jf} \leq 1, (i, j) \in E \end{cases}$$

L'ultima condizione indica che le frequenze sono diverse tra vicini.

6 Partizione di sottoinsiemi

6.1 Problema di partizione

6.1.1 Descrizione del problema

Sia $B = \{1, \ldots, n\}$ l'insieme delle barche.
Definiamo F come la famiglia di tutti i sottoinsiemi $S \subseteq B$ tali che:

- Le barche non si sovrappongono fra loro per questioni di orario (possono essere assegnate allo stesso molo)

Ogni sottoinsieme rappresenta un possibile **turno** occupato da un singolo molo.
Per ogni S, introduciamo la variabile binaria:

$$y_s = \Big\{1, \text{ se apriamo/usiamo un molo con il gruppo di barche S0}$$

Bisogna trovare la famiglia ottimale di sottoinsiemi compatibili.

6.1.2 Inizializzazione

Variabili:

- P = insieme degli indici degli insiemi di turni compatibili.
- $a_{ip} \in \{0, 1\}$ parametro binario, 1 se la barca $i \in B$ è contenuta nel turno $p \in P$.
- $y_p \in \{0, 1\}$, 1 se si usa p.

Funzione obiettivo, andiamo a formulare la partizione dei sottoinsiemi:

$$minZ = \sum_{S \in F} y_s$$

Vincoli:

- Ogni barca è associata a un molo solo:

$$\sum_{S \in F : i \in S} y_s = 1$$

- Variabile binaria:

$$y_s \in \{0, 1\} \forall S \in F$$

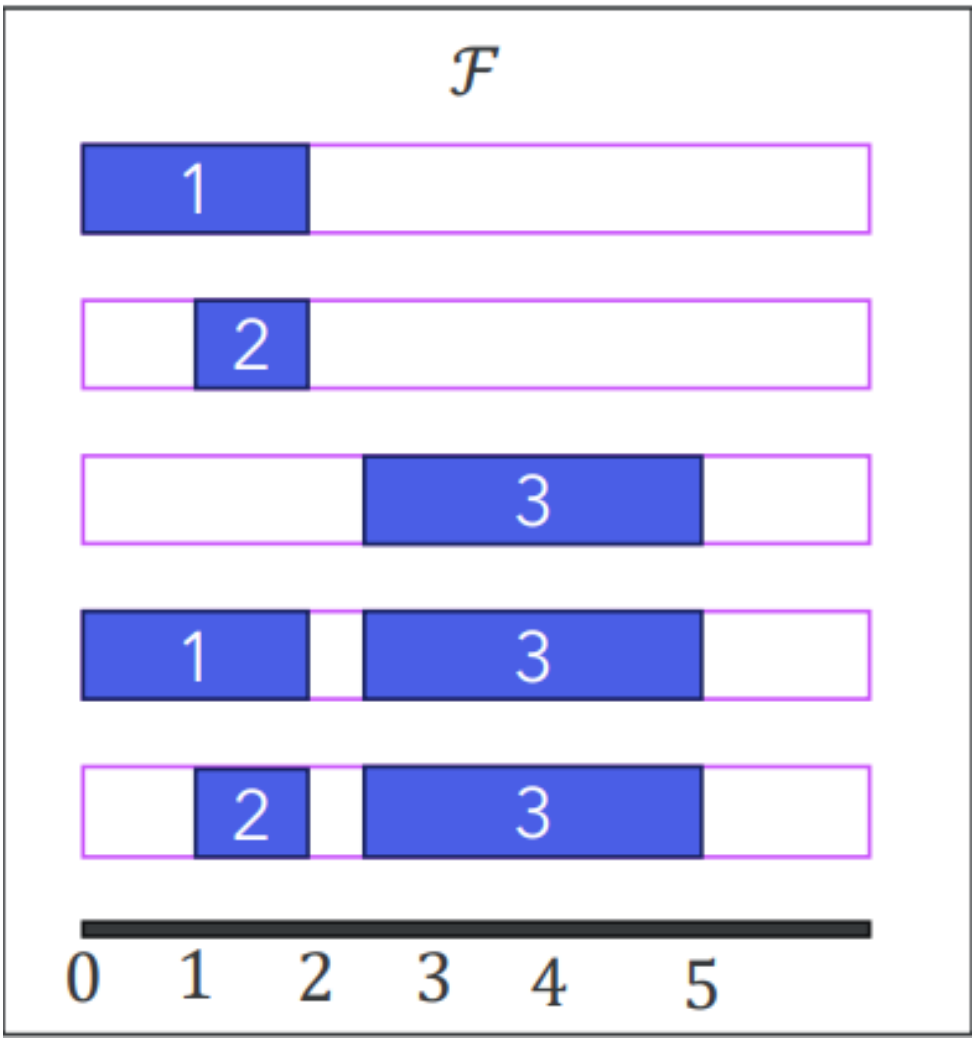


Figure 2: Partizioni delle barche

A questo punto si potrebbe sviluppare una matrice per capire le partizioni:

P (programmi) e corrispondenza con $\mathcal{F}$

Definiamo $P = \{p_1, p_2, p_3, p_4, p_5\}$

con

$S_{p_1} = \{1\}, \quad S_{p_2} = \{2\}, \quad S_{p_3} = \{3\},$

$S_{p_4} = \{1,3\}, \quad S_{p_5} = \{2,3\}.$

Matrice di a_{ip} (righe = barche i, colonne = programmi p)

$$(a_{ip}) = \begin{array}{c|ccccc} & p_1 & p_2 & p_3 & p_4 & p_5 \\ i=1 & 1 & 0 & 0 & 1 & 0 \\ i=2 & 0 & 1 & 0 & 0 & 1 \\ i=3 & 0 & 0 & 1 & 1 & 1 \end{array}$$

Figure 3: Tabella delle partizioni

6.2 Problema di configurazione

6.2.1 Descrizione del problema

Tommaso ha un ristorante e va al mercato per comprare dal macellaio e dall'ortofruttivendolo.

Cassa Proteica (M) – dalla macelleria di nonno Rinaldo. Ogni cassa contiene:	• 1 porzione di filetto M_f (taglio pregiato, perfetto per le grigliate) • 1 porzione di avanzo/ossa M_a (per brodi e stufati) • Prezzo: **6,5 €**.
Cassa Orto (O) – dall'orto di Marina. Ogni cassa contiene:	• 1 mazzo di foglie miste O_i (insalate, rucola, etc.) • 1 porzione di radici O_r (patate, barbabietole, carote) • Prezzo: **4,0 €**.
Barattolo Salse (S) – produzione casalinga di conserve: ogni unità è	• 1 barattolo di salsa/stufato S (ideale per condire o come base). • Prezzo: **1,5 €**

Figure 4: Tabella dei piatti

Tommaso ha 16.5 euro a disposizione e vuole decidere quanti casse/barattoli comprare e quali piatti inserire nel menu.
Ogni piatto richiede dei componenti base (tabella).
Ogni piatto ha un valore di gradimento dei clienti.
Se il menu ha almeno 4 piatti allora Tommaso ha +10 punti di valore di gradimento (menu vario).

Piatto (c)	M_f	M_a	O_i	O_r	S	Valore v_c
1 – Tagliata con insalata	1	1	1	0	0	6
2 – Stufato rustico	1	1	0	0	1	3
3 – Filetto al sale	1	0	0	0	0	2
4 – Insalata di carne	1	0	1	0	1	5
5 – Filetto con contorno	1	1	1	1	0	9
6 – Stufato classico	1	1	0	0	0	10
7 – Salsa del giorno	0	0	0	0	1	7
8 – Piatto gourmet	1	1	1	1	1	8

Figure 5: Tabella degli ingredienti

Variabili:

$$x_c \in \{0,1\}, \qquad c \in C = \{1,\dots,8\}$$

dove $x_c = 1$ se il piatto c è inserito nel menù.

$$y_M, y_O, y_S \in \mathbb{N}_0$$

indicano rispettivamente il numero di Casse Proteiche, Casse Orto e Barattoli Salse acquistati.

$$z \in \{0,1\}$$

vale 1 se il menù contiene almeno 4 piatti.

Funzione obiettivo:

$$maxW = 6x_1 + 3x_2 + 2x_3 + 5x_4 + 9x_5 + 10x_6 + 7x_7 + 8x_8 + 10z$$

Vincoli:

$$\begin{cases} z \in \{0,1\} \\ x_c \in \{0,1\}, \forall c \in C \\ 4z - (x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_7 + x_8) \leq 0 \\ y_M - (x_1 + x_2 + x_3 + x_4 + x_5 + x_6 + x_8) \geq 0 \\ y_M - (x_1 + x_2 + x_5 + x_6 + x_8) \geq 0 \\ y_S - (x_2 + x_4 + x_7 + x_8) \geq 0 \\ y_O - (x_1 + x_4 + x_5 + x_8) \geq 0 \\ y_O - (x_5 + x_8) \geq 0 \end{cases}$$

Sapendo che $a_{h,c} \in \{1(\textit{piatto c usa h}), 0\}$, $h \in H = \{M_f, M_b, O_i, O_r, S\}$, $d_{h,j} \in \{1(\textit{h nella cassa j}), 0\}$ e che $j \in \{M, O, S\}$ allora stabiliamo il vincolo della disponibilità degli ingredienti:

$$\sum_{c \in C} a_{h,c} x_c \leq \sum_{j \in J} d_{h,j} y_j \qquad \forall h \in H$$

Infine, ecco il vincolo di budget:

$$6,5 \cdot y_M + 4,0 \cdot y_O + 1,5 \cdot y_S \leq 16,5$$

6.3 Problema di instradamento

6.3.1 Descrizione del problema

In molti casi la famiglia di interesse non è direttamente fornita in input, ma deve essere calcolata a partire da condizioni logiche sui sottoinsiemi ammissibili, come nell'esempio seguente. La Trans-Port (TP), azienda logistica che lavora all'Interporto di Guasticce, deve pianificare i viaggi dei suoi camion.

- Ciascun camion parte dal **deposito** (d) e deve consegnare containers pieni provenienti dal porto agli **importatori** (I) e/o ritirare containers vuoti dagli **esportatori** (E) per riportarli a d, da dove vengono poi trasportati al porto per essere riutilizzati; le tratte tra il porto e d sono ovviamente fisse e comunque organizzate indipendentemente, quindi non devono essere considerate.
- Ciascun camion porta fino a due container, e può essere usato sia per la consegna che per il ritiro; per motivazioni organizzative, però, deve prima consegnare (a meno che non parta vuoto) e poi ritirare (a meno che non rientri vuoto).
- È nota la **distanza** c_{hk} tra tutte le coppie h, k di punti rilevanti (d, tutti gli $i \in I$ e tutti gli $e \in E$), che può essere assunta simmetrica.
- È anche nota la **domanda** q_i, $i \in I$, e q_e, $e \in E$, di containers da consegnare/ritirare per ogni esportatore/importatore; si noti che un certo esportatore i potrebbe anche essere un importatore e, ma questo si modella semplicemente considerandoli diversi e ponendo $c_{ie} = 0$.
- Si può assumere che la TP disponga di un numero illimitato di camion (una volta rientrato al deposito un camion può iniziare un altro viaggio, e la durata dei viaggi è tale per cui i camion effettivamente disponibili sicuramente riusciranno a fare tutti i viaggi necessari entro la giornata lavorativa).
- Occorre formulare come PLI il problema che la TP deve risolvere per soddisfare le domande di tutti gli esportatori/importatori minimizzando la distanza totale percorsa dai suoi camion, e quindi il costo di trasporto.

6.3.2 Inizializzazione

Per modellare il problema definiamo, a partire dai dati in input, quattro insiemi di cammini:

P1	Tutti i cammini che visitano un solo utente: • $d \to i \to d$ per $i \in I$ • $d \to e \to d$ per $e \in E$	
P2	Tutti i cammini che visitano esattamente due utenti: • $d \to i_1 \to i_2 \to d$ per $i_1, i_2 \in I$ • $d \to i \to e \to d$ per $i \in I, e \in E$ • $d \to e_1 \to e_2 \to d$ per $e_1, e_2 \in E$	Nota: i cammini in cui si visita prima un esportatore di un importatore sono esclusi per la politica aziendale.
P3	Tutti i cammini che visitano esattamente tre utenti: • $d \to i_1 \to i_2 \to e \to d$ per $i_1, i_2 \in I$ ed $e \in E$ • $d \to i_1 \to e_1 \to e_2 \to d$ per $i \in I$ ed $e_1, e_2 \in E$	Nota: non si possono visitare tre importatori o esportatori per la capacità del camion, e no si può visitare prima un esportatore e poi un importatore)
P4	Tutti i cammini che visitano esattamente quattro utenti: • $d \to i_1 \to i_2 \to e_1 \to e_2 \to d$ per $i_1, i_2 \in I$ ed $e_1, e_2 \in E$.	

Figure 6: Tabella descrittiva

Definiamo:

- L'insieme dei cammini: $P = P1 \cup P2 \cup P3 \cup P4$
- Il costo del cammino c_p: $p \in P$

Si nota che nulla impone che $i_1 \neq i_2$ o che $e_1 \neq e_2$, un camion può effettuare entrambi le operazioni di carico/scarico allo stesso operatore. Stabiliamo quindi:

$$a_{ip} = \{0, 1, 2\} \text{ numero di volte che il cammino p visita l'importatore i}$$

$$a_{ep} = \{0, 1, 2\} \text{ numero di volte che il cammino p visita l'esportatore e}$$

Come ultima variabile introduciamo una contatore:

$$y_p = \text{ numero di camion che effettua il cammino } p \in P$$

Funzione obiettivo:

$$minZ = \sum_{p \in P} C_p \cdot y_p$$

Vincoli:

$$\begin{cases} y_p \in \mathbb{N}, \forall p \in P \\ \sum_{p \in P} a_{ip} \cdot y_p = q_i, \forall i \in I \\ \sum_{p \in P} a_{ep} \cdot y_p = q_e, \forall e \in E \end{cases}$$

7 Problemi su reti: problemi di flusso e di cammino

Finora abbiamo utilizzato variabili logiche e intere, ma anche le variabili **continue** possono assumere un valore logico, basta che si rappresentino cosi:

$$k = \begin{cases} 0, \text{ falso} \\ > 0, \text{ vero, indicandone allo stesso tempo l'intensità} \end{cases}$$

Queste variabili sono fondamentali per analizzare i problemi di **flusso della rete**.
Vediamo anche le variabili **quantitative**:

- Descrivono quanto viene trasportato lungo gli archi di un grafo.
- Stabiliscono implicitamente il collegamento effettuato.

I problemi di flusso sono applicabili a situazioni reali, come la ditribuzione di gas, elettricità, acqua e altri impianti.

7.1 Problema delle fognature (esempio)

7.1.1 Descrizione del problema

In una città suddivisa in 4 quartieri, identificati dai vertici

$$V = \{1, 2, 3, 4\},$$

Ogni quartiere $i \in V$ produce una quantità nota di acque reflue (nell'immagine accanto al nodo), espressa in m^3/h. Le acque reflue devono essere convogliate a un unico depuratore, rappresentato dal nodo 5. Per realizzare la rete fognaria, disponiamo di un insieme di condotte candidate, rappresentate dagli archi orientati del grafo:

$$A = \{(1,5), (2,1), (2,3), (2,5), (3,5), (4,3)\}.$$

Il costo associato a ciascun arco $(i, j) \in A$ è proporzionale al flusso di acque reflue trasportato e viene indicato con

$$c_{ij} \quad [\text{costo per } \text{m}^3/\text{h}].$$

L'obiettivo è determinare quali condotte attivare e come dimensionarle, ossia quali flussi assegnare agli archi, in modo da trasportare l'intera produzione dei quartieri al depuratore, minimizzando il costo complessivo. Il problema può essere formulato come un problema di flusso, noto come *Problema di Disegno di Rete*.

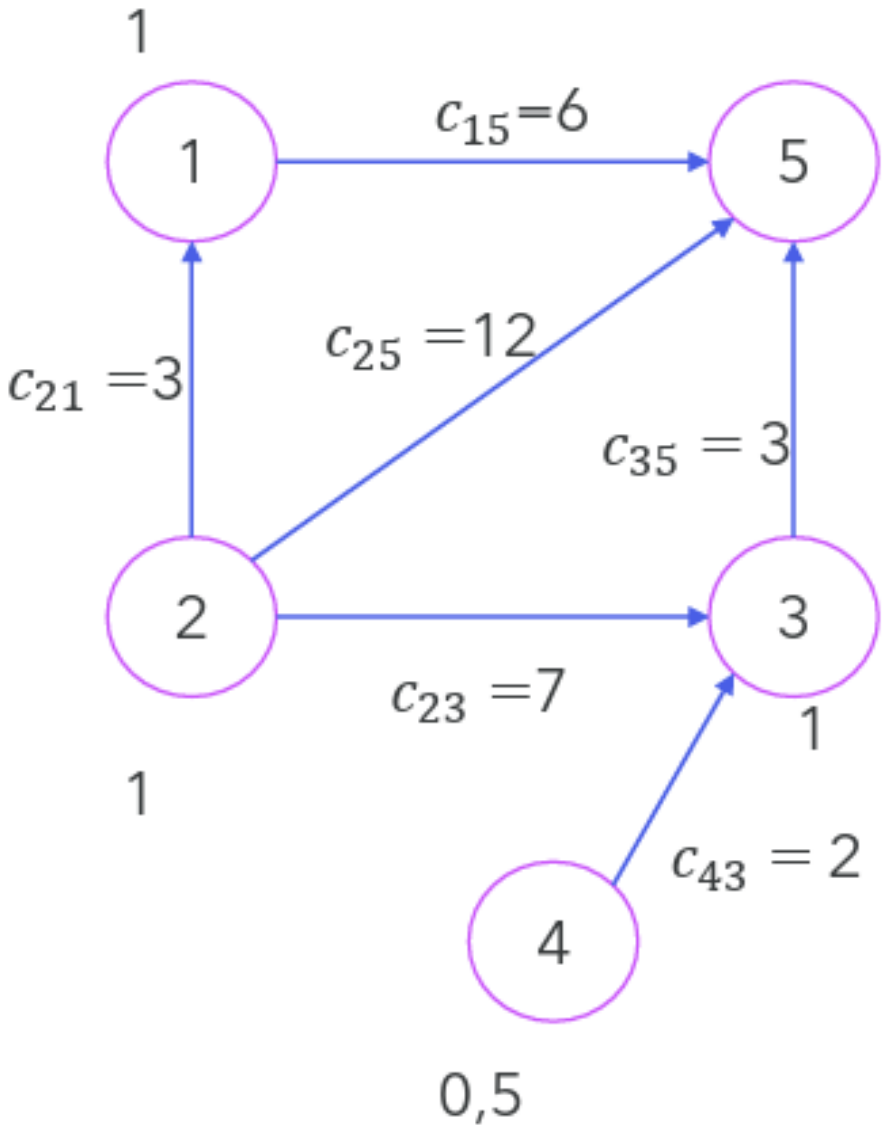


Figure 7: Grafo associato

7.1.2 Inizializzazione

Variabili:

- x_{ij}: flusso su un arco $(i, j) \in A$ (dimensione condotta)

Funzione obiettivo:

$$minZ = 6x_{15} + 3x_{12} + 7x_{23} + 12x_{25} + 3x_{35} + 12x_{43}$$

$$minZ = \sum_{(i,j)\in A} c_{ij} \cdot x_{ij}$$

Per i vincoli bisogna tenere di conto che ogni flusso che entra è uguale a quello che esce:

- Nodo1: $x_{21} + 1 = x_{15}$
- Nodo2: $1 = x_{21} + x_{23} + x_{25}$

- Nodo3: $x_{23} + x_{43} + 1 = x_{35}$
- Nodo4: $0,5 = x_{43}$
- Dominio: $x_{ij} \geq 0, \forall (ij) \in A$

Per il quinto nodo non serve dato che sarebbe ridondante.
Inoltre, ecco un altro vincolo importante, ovvero quello di **bilancio di flusso:**

$$\sum_{j:(j,i)} x_{ji} - \sum_{j:(i,j)} X_{ij} = b_i \forall i \in V$$

Questo fa la sottrazione tra i flussi che entrano ed escono dal nodo i:

- $b_i < 0$: il nodo è sorgente.
- $b_i > 0$: il nodo è pozzo.
- $b_i = 0$: il nodo è di trasferimento.

7.2 Problema del flusso di costo minimo

7.2.1 Descrizione del problema

Gneneralizziamo un problema di flusso: il problema è definito su un grafo orientato $G = (V, A)$.
A ciascun nodo $i \in V$ è associato un valore b_i che rappresenta il **bilancio di flusso**.

- $b_i < 0$: il nodo è sorgente.
- $b_i > 0$: il nodo è pozzo.
- $b_i = 0$: il nodo è di trasferimento.

Per ogni arco $(i, j) \in A$ è dato un costo c_{ij} che rappresenta il costo per unità di flusso che transita sull'arco.
Per ogni arco è data anche una capacità $\mu_{ij} \in \mathbb{R}^+$ che rappresenta il massimo flusso ammissibile sull'arco.
Determinare i flussi in modo da:

- Soddisfare i bilanci di flusso in ogni nodo
- Rispettare i vincoli di capacità su ogni nodo
- Minimizzare il costo totale

7.2.2 Inizializzazione

Variabili:

$$x_{ij} = \text{flusso sull'arco } (i, j) \in A$$

Funzione obiettivo:

$$minZ = \sum_{(i,j) \in A} c_{ij} \cdot x_{ij}$$

Vincoli:

- Vincoli di bilancio di flusso:

$$\sum_{j:(j,i)} x_{ji} - \sum_{j:(i,j)} X_{ij} = b_i \forall i \in V$$

- Vincoli di capacità

$$0 \leq x_{ij} \leq \mu_{ij} \forall (i,j) \in A$$

7.3 Problema del cammino minimo

7.3.1 Descrizione del problema

Abbiamo un grafo $G = (V, A)$, per ogni arco $(i, j) \in A$ è dato un costo c_{ij}.
Esiste un singolo nodo sorgente e un singolo pozzo. Trovare il cammino minimo dalla sorgente (s) al pozzo (p).

7.3.2 Inizializzazione

Il cammino minimo è un particolare problema di flusso, ma bisogna settare i flussi a valori binari
Variabili:

$$x_{ij} : \text{flusso che scorre sull'arco } (i,j) \in A$$

$$c_{ij} : \text{costo del flusso sull'arco } (i,j) \in A$$

Funzione obiettivo:

$$\sum_{(i,j)\in A} c_{ij} \cdot x_{ij}$$

Vincoli:

$$\begin{cases} \sum_{j:(j,i)} x_{ji} - \sum_{j:(i,j)} x_{ij} = b_i \forall i \in V \\ 0 \leq x_{ij} \leq \mu_{ij}, \forall (i,j) \in A \end{cases}$$

8 Variabili a valori discreti

8.1 Problema delle fognature ampliato

8.1.1 Descrizione del problema

Torniamo al problema della fognatura.
In realtà, le tubature fognarie possono avere solo valori specifici di portate.
La portata della condotta non può essere supposta di essere uguale al flusso, potrebbe infatti essere necessario installare condotte piu grosse.
Supponiamo che questi siano i soli 3 valori delle dimensioni:

- 0,7
- 1,4
- 3

Come facciamo a modellarlo?

8.1.2 Inizializzazione

Partiamo dalle variabili:

$$x_{ij} : \text{flusso che scorre sull'arco } (i,j) \in A$$

$$y_{ij} : \text{dimensione della condotta sull'arco } (i,j) \in A$$

La variabile y_{ij} è detta **discreta**, ovvero che assume solo specifici valori e non tutti i valori reali.
Funzione obiettivo:

$$\sum_{(i,j)\in A} c_{ij} \cdot y_{ij}$$

Associamo, come vincoli, delle espressioni binarie:

$$y_{ij} = 0,7z^1 + 1,4z^2 + 3z^3$$

$$z^1 + z^2 + z^3 \leq 1$$

$$z^h \in \{0,1\}$$

Una condotta posso non utilizzarla, ma se la uso solo una dimensione può essere scelta.

9 Variabili semicontinue

Ricordiamo il problema della fognatura ampliato.
Immaginiamo che, se decido di costruire la condotta $(i, j) \in A$, la sua dimensione dovrà essere in un intervallo $[l_{ij}, u_{ij}]$.
Adesso i vincoli diventano:

$$\begin{cases} z^h \in \{0, 1\} \\ y_{ij} \in \{0\} \cup [i_{ij}, u_{ij}] \end{cases}$$

In questo caso y_{ij} è una variabile **semicontinua**.
Queste vengono utilizzate spesso in problemi di produzione dove i prodotti devono mantenere certi intervalli.

Addirittura, questo tipo di variabili possono esprimere intervalli disgiunti, in tal caso si fa affidamento a più variabili booleane, la cui combinazione indica in quale intervallo ci troviamo:

$$\begin{cases} z^1, z^2, z^3 \in \{0, 1\} \\ (intervallo1) \cdot z^1 \\ (intervallo2) \cdot z^2 \\ (intervallo3) \cdot z^3 \\ \sum_{i=1}^{3} z^h \leq 1 \end{cases}$$

10 Costi di setup

In un problema di produzione è tipico avere dei costi di produzione (c) e dei **costi di setup** (k).
I costi di setup dipendono dall'attivazione della produzione.
Inoltre, esiste una capacità minima (l) e massima (u) di produzione.
Vediamo la funzione obiettivo:

$$minZ = cx + ky$$

Vincoli:

$$ly \leq x \leq uy$$

$$x \geq 0$$

$$y \in \{0, 1\}(\text{se produco o meno})$$

Nel primo vincolo è gia inclusa la condizione della legatura tra x e y (se produco, y = 1).

11 Massimizzare minimi e minimizzare massimi

11.1 Tempi di completamento delle barche (esempio)

11.1.1 Descrizione del problema

Una compagnia logistica ha n barche $B = \{1, \dots, n\}$ (tutte immediatamente pronte in rada) che devono essere assegnate ai moli disponibili $M = \{1, \dots, m\}$ per essere scaricate. Ogni barca $i \in B$ richiede un tempo di servizio p_i per lo scarico. Una volta che una barca è attraccata al molo deve restarvi fino al completamento dello scarico. Un molo può servire una sola barca alla volta.
Le barche attendono fino a che non vengono impegnate. L'obiettivo è minimizzare il **tempo di completamento** (makespan), ovvero il tempo in cui tutte le barche finiscono di scaricare.

11.1.2 Inizializzazione

Insiemi:

$$B = \{1, \dots, n\}$$
$$M = \{1, \dots, m\}$$

Variabili:

$$p_i = \text{tempo di impiego della barca } i \in B$$
$$a_{bm} = \text{binaria, assegnazione barca-molo}$$

Funzione obiettivo:

$$\sum_{i \in B} p_i \cdot a_{bm}$$

Vincoli:

$$\begin{cases} p_i \geq 0 \\ a_{bm} = \{0, 1\} \\ \sum_{b \in B} a_{bm} \leq 1, \forall m \in M \\ \sum_{m \in M} a_{bm} \leq 1, \forall b \in B \end{cases}$$

Mettiamo in atto questo per aiutarci:

$$t_m = \text{tempo completamento di un molo}$$

$$\sum_{i \in B} p_i \cdot a_{bm} = t_m, \forall m \in M$$

Dobbiamo minimizzare il massimo:

$$min(max(t_m1, t_m2, \dots, t_mm))$$

Si minimizza il massimo della funzione obiettivo.
Se avessimo dovuto invece fare il contrario, ovvero massimizzare il minimo, il processo era analogo:

$$max(min(t_m1, t_m2, \dots, t_mm))$$

12 Linearizzazione valore assoluto

12.1 Equipartizione (esempio)

12.1.1 Descrizione del problema

Due fratelli devono spartirsi equamente l'eredità composta da $n \in N$ beni indivisibili. Ogni bene ha un valore $a_i \in \mathbb{R}$.
Vogliamo suddividere i beni in due insiemi diversi minimizzando la differenza di valore.

12.1.2 Inizializzazione

Insiemi:

$$N = \{1, \ldots, n\} A = \text{Insieme primo fratello}$$

Variabili:

$$a_i \in R$$

$$x_i = \begin{cases} 1, \text{assegnato al fratello 1} \\ 0, \text{assegnato al fratello 2} \end{cases}$$

Funzione obiettivo, valori dei beni del primo fratello meno quelli del secondo:

$$minZ = |\sum_{i \in N} a_i \cdot x_i - \sum_{i \in N} a_i \cdot (1 - x_i)|$$

Vincoli:

$$\begin{cases} x_i = \{0, 1\} \end{cases}$$

Come possiamo vedere la funzione obiettivo non è lineare, per risolvere consideriamo che:

$$|x| = max(x, -x)$$

Per minimizzarlo, facciamo:

$$min(max(x, -x))$$

Usiamo una variabile "v" ausiliaria e scriviamo il modello lineare:
Prendiamo il minimo tra i vari v che:

$$v \geq \sum_{i \in N} a_i \cdot x_i - \sum_{i \in N} a_i \cdot (1 - x_i)$$

$$v \geq \sum_{i \in N} a_i \cdot (1 - x_i) - \sum_{i \in N} a_i \cdot x_i$$

13 Vincoli disgiuntivi

Solitamente, in Programmazione Lineare, l'insieme ammissibile è descritto dall'intersezione di sotto-piani legati a **vincoli**.
Questo piano risultante si chiama **poliedro convesso**.
In altri casi ci può essere richiesto di realizzare **casi distinti** e quindi non bisogna trovare un'intersezione, in quanto la soluzione potrebbe essere in un piano o in un altro.
Si introducono variabili binarie per specificare se un vincolo può essere violato o meno, garantendo che non tutti i vincoli (o uno specifico numero di vincoli) non siano violati contemporaneamente.
Per farlo, si usa un valore speciale:

$$M = \text{Valore reale molto grande}$$

Considerando due variabili binarie a e b, i vincoli di maggioranza e minoranza diventano cosi:

$$expression \leq k \cdot Ma$$

$$expression \geq k \cdot -Mb$$

Se un vincolo viene negato (e quindi la variabile binaria = 1) allora viene reso inutile dal valore M.

13.1 Grafo spazio temp e probabilità (esempio)

13.1.1 Descrizione del problema

Il prof. Viaggioni (PV) alla fine di un'importante conferenza si trova bloccato a causa di uno sciopero che coinvolge tutti i mezzi di trasporto. PV deve quindi trovare un modo per tornare a casa ed applica le sue conoscenze: Utilizziamo un grafo **spazio-tempo** $G = (N, A)$.

- I **nodi** sono coppie (luogo,orario), a partire da quelli della forma (Conf, t) per tutti i momenti t in cui un mezzo dovrebbe partire dalla città sede della conferenza.
- Gli **archi** "viaggio" $(i, j) = ((s_i, t_i), (s_j, t_j)) \in A$ rappresentano le corse previste in partenza da s_i al tempo t_i ed in arrivo al luogo $s_j \neq s_i$ al tempo $t_j > t_i$.
- Sono presenti anche archi **attesa**, in cui $s_i = s_j$ (e $t_j > t_j$), che rappresentano lo stare fermi in attesa di una coincidenza.
- A ciascun arco è associato:
  - Il costo monetario $c_{ij} > 0$ (tipicamente sugli archi attesa, tranne quelli dove bisogna sostare in albergo)
  - La probabilità p_{ij} che l'arco sia effettivamente percorribile (< 1 sugli archi attesa se la coincidenza è "stretta" o gli alberghi in s_i sono "pieni").

Il PV ha un budget monetario $C > 0$ per pagarsi il viaggio e vuole che la probabilità di successo del viaggio sia almeno del 90% complessivamente.
Si aiuti il PV a tornare a casa (probabilmente) scrivendo come PLI il problema di determinare gli spostamenti da fare, minimizzando il tempo totale del viaggio.

13.1.2 Modellazione

Variabili (oltre a quelle definite gia):

$$x_{ij} = \begin{cases} 1, \text{ se uso arco } (i,j) \in A \\ 0 \end{cases}$$

Funzione obiettivo:

$$minZ = \sum_{(i,j)\in A: j=(casa,t)} t \cdot x_{ij} - \sum_{(i,j)\in A: t=(Conf,t)} t \cdot x_{ij}$$

Vincoli:

$$\begin{cases} \sum_{(i,j)\in A} c_{ij} \cdot x_{ij} \\ x_{ij} \in \{0,1\}, (i,j) \in A \\ \\ \sum_{j:(j,i)\in A} x_{ji} - \sum_{j:(i,j)\in A} x_{ij} = \begin{cases} -1, i = s \\ 1, i = d \\ 0, i \neq s, d \end{cases} \\ \\ \sum_{(i,j)\in A} c_{ij} \cdot x_{ij} \leq c \end{cases}$$

Vogliamo scrivere il vincolo sulle probabilità:

$$\prod_{(i,j)\in A: x_{ij}=1} p_{ij} \geq 0,9$$

$$\log\left(\prod_{(i,j)\in A: x_{ij}=1} p_{ij}\right) \geq \log(0,9)$$

$$\sum_{(i,j)\in A: x_{ij}=1} \log(p_{ij}) \geq \log(0,9)$$

$$\sum_{(i,j)\in A} x_{ij} \cdot \log(p_{ij}) \geq \log(0,9)$$

14 Grafi e reti di flusso

14.1 Definizioni iniziali

Una **rete** è un grafo orientato pesato $G = (N, A)$ con:

$$n = |N|$$

$$m = |A|$$

Gli archi sono canali di flusso e possono essere:

- **Discreti**: numero di auto.
- **Continui**: quantità di petrolio in un oleodotto.

Ad ogni nodo è associato un **valore reale** b_i detto **deficit**:

$$b_i > 0, \text{ Quantità di bene che esce dalla rete, nodo di destinazione, pozzo o output}$$

$$b_i < 0, \text{Quantità di bene che entra nella rete, nodo di origine, sorgente o input}$$

$$b_i = 0, \text{Nodo di trasferimento}$$

Nei problemi di flusso di solito:

$$\text{Domanda globale} = \text{Offerta globale}$$

Quindi:

$$D = \{i \in N : b_i > 0\}$$
$$O = \{i \in N : b_i < 0\}$$
$$\sum_{i \in D} b_i = -\sum_{i \in O}^{b_i}$$
$$\sum_{i \in N} b_i = 0$$

Ad ogni arco $a_k = (i, j)$ sono associati:

- **Costo:** c_k o c_{ij}, costo per unità di bene che attraversa l'arco.
- **Capacità inferiore:** l_k o l_{ij}, minimo numero di unità trasportabili.
- **Capacità superiore:** u_k o u_{ij}, massimo numero di unità trasportabili.

Spesso $l_k = 0$ e viene considerata solo la capacità superiore.
Un vettore $x = [x_{ij}]_{(i,j)\in A}$ rappresenta un **flusso su G** se soddisfa i vincoli di conservazione del flusso:

$$\sum_{(j,i)\in BS(i)} x_{ji} - \sum_{(i,j)\in FS(i)} x_{ij} = b_i, \forall i \in N$$

Con:

$$BS = \text{archi entranti in i}$$

$$FS = \text{archi uscenti da i}$$

Un **flusso ammissibile** rispetta i vincoli di ammissibilità:

$$l_{ij} \leq x_{ij} \leq u_{ij}, (i,j) \in A$$

Ad ogni flusso x è associato un **costo totale**:

$$c(x) = \sum_{(i,j)\in A} c_{ij} \cdot x_{ij}$$

Alcuni problemi possono essere risolti riconducendoli a problemi di Flusso di Costo Minimo (MCF).

14.2 schedulazione della produzione (esempio)

14.2.1 Descrizione

L'industria dolciaria PereCani, nota produttrice di panettoni, deve decidere come utilizzare al meglio, nel corso dell'anno, la sua linea di produzione per tali dolci. La PereCani conosce una stima $b_i, i = 1, \ldots, 12$, del numero di panettoni che venderà ogni mese dell'anno. Il costo unitario di produzione e la massima quantità di panettoni producibile in un mese variano anch'esse, a secibda du akcybu fattori quali il prezzo delle materie prime e la disponibilità di personale impegnato anche su altre linee di produzione, e anche di esse si hanno le stime, rispettivamente c_i e u_i con $i = 1, \ldots, 12$. I panettoni prodotti al mese i possono essere venduti immediatamente, oppure immagazzinati per essere poi venduti nei mesi successivi: il magazzino ha una capacità massima di U, ed un costo unitario di immagazzinamento pari a C. All'inizio il magazzino contiene b_0 panettoni, e si desidera che alla fine dell'anno ne contenga b_13.

14.2.2 Inizializzazione

Variabili:

- x_i: quantità di panettoni prodotti nel mese i
- s_i: quantità di panettoni immagazzinati alla fine del mese i

Funzione obiettivo, minimizzare il costo totale di produzione e magazzino:

$$minZ = \sum_{i=1}^{12} (c_i \cdot x_i + C \cdot s_i)$$

Vincoli:

$$\begin{cases} i = 1, \ldots, 12 \\ s_{i-1} + x_i - s_i = b_i, \text{Bilancio del magazzino} \\ 0 \leq x_i \leq u_i, \text{capacità di produzione} \\ 0 \leq s_i \leq U, \text{capacità del magazzino} \end{cases}$$

Un problema del genere può essere definito come un problema di **flusso di costo minimo**:

$$minC(x)|Ex = b, l \leq x \leq u$$

Dove:

$$Ex = b: \text{ vincoli di conservazione del flusso in forma vettoriale}$$

$$l \leq x \leq u: \text{ vincoli di capacità degli archi}$$

Ecco come si imposta questo problema, definito anche come **Lot Sizing**, trasformandolo in un problema tale:

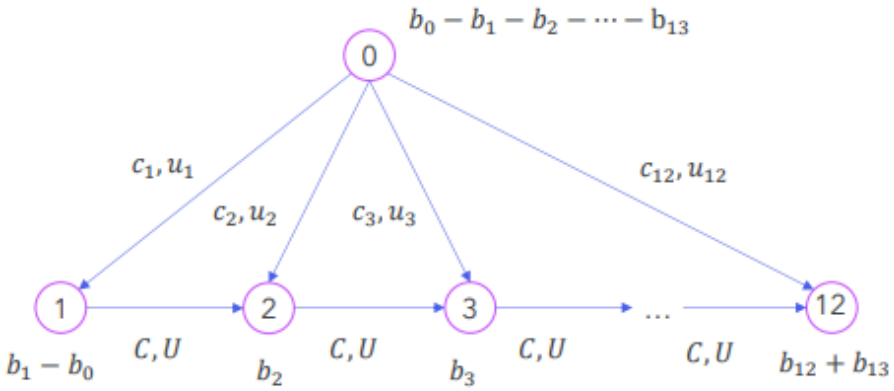


Figure 8: Lot Sizing Problem

- Gli archi dal nodo fittizio 0 agli altri nodi rappresentano la produzione.
- Gli archi di tipo $(i, i+1)$ rappresentano il magazzino.
- Al nodo 1 sono sottratti i panettoni gia presenti in magazzino.
- Al nodo 12 sono sottratti i panettoni che dovranno essere lasciati in magazzino.
- Il bilancio al nodo 0 è la produzione totale di panettoni durante l'anno.

14.3 Trasformazioni equivalenti

Molti problemi in realtà possono essere definiti nel solito modo, ovvero come **problemi di flusso di costo minimo (MCF)**.
Su tali problemi si possono fare alcune assunzioni che ne semplificano la descrizione e l'analisi.
Se l'istanza originale non li rispetta, si possono effettuare delle trasformazioni equivalenti.
Iniziamo con il vedere le assunzioni:

- Singola sorgente - singolo pozzo
- Capacità inferiori nulle
- Nessuna capacità associata ai nodi
- Nessun eccesso di domanda o di offerta

14.3.1 Sorgenti e pozzi multipli

Per trasformare un'istanza dall'avere sorgenti e pozzi multipli ad un problema che ha una sola sorgente e un solo pozzo si introduce una **rete ampliata** $G^I = (N^I, A^I)$.

Nodi fittizi : $N^I = N \cup \{s, t\}$ con:

- **s:** super sorgente con offerta $b_s = \sum_{j \in O} b_j$
- **t:** super pozzo con domanda $b_t = \sum_{i \in D} b_i$
- Tutti gli altri nodi sono di trasferimento

Archi fittizi :

- (s, j) per $j \in O$, con capacità $u_{sj} = -b_j$ e costo 0
- (i, t) per $i \in D$, con capacità $u_{it} = b_i$ e costo 0

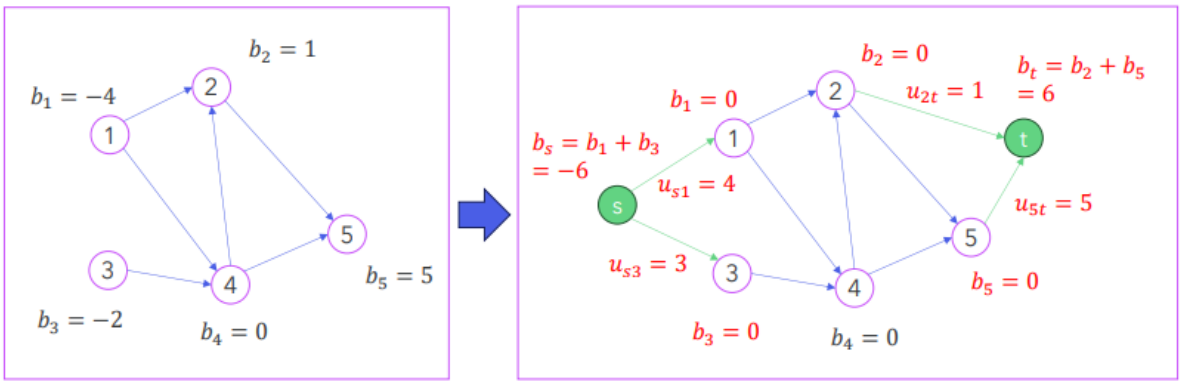


Figure 9: Esempio di trasformazione: sorgenti e pozzi

14.3.2 Capacità inferiore non nulle

Se un arco (i, j) ha capacità minima $l_{ij} \neq 0$, il flusso x_{ij} deve soddisfare $x_{ij} \geq l_{ij}$.
Per eliminare l_{ij} si procede cosi:

Denominazione del nuovo flusso :

$$x^I_{ij} = x_{ij} - l_{ij}$$

Aggiornamento dei parametri :

$$b^I_i = b_i + l_{ij}$$

$$b^I_j = b_i - l_{ij}$$

$$u^I_{ij} = u_{ij} - l_{ij}$$

Inoltre, la funzione obiettivo viene integrata con un costo fisso:

$$obj + c_{ij} \cdot l_{ij}$$

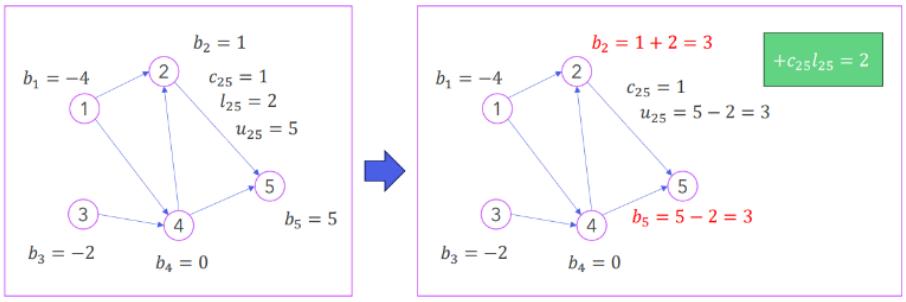


Figure 10: Esempio di trasformazione: capacità inferiori

14.3.3 Limiti di capacità sui nodi

In molti casi un nodo i può avere **capacità massima** e **minima**. Questi non sono gestibili dal modello del flusso di costo minimo e vanno eliminati.
Si eliminano introducendo due nodi e un arco fittizio:

- Il nodo i viene sostituito con due nodi i^I di ingresso e i^{II} di uscita.
- Gli archi entranti in i diventano (j, i^I).
- Gli archi uscenti da i diventano (i^{II}, j).
- Si aggiunge un arco fittizio (i^I, i^{II}) con:

  costo $c_{i^I i^{II}} = 0$.

  capacità $[l_i, u_i]$.

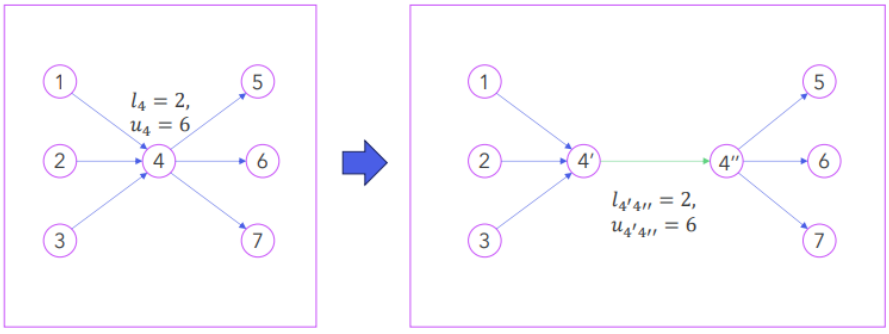


Figure 11: Esempio di trasformazione: rimozione limiti di capacità

14.3.4 Eccesso di domanda o offerta

Spesso la quantità disponibile nei nodi $i \in O$ rappresenta una **massima offerta possibile** che non necessariamente va immessa tutta nella rete.
Il vincolo di conservazione del flusso diventa quindi una disuguaglianza:

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} \geq b_i, i \in O$$

dove $b_i < 0$ rappresenta l'offerta massima del nodo i.
Per risolvere, si costruisce una rete ampliata $G^I = (N^I, A^I)$ con una sorgente fittizia:

$$N^I = N \cup \{s\}$$

Nuovi bilanci:

$$b_s = -\sum_{j \in D} b_j$$

$$b_i = 0, i \in O$$

Nuovi archi:

$$(s, i), i \in O$$

con costo $c_{si} = 0$ e capacita $u_s i = -b_i$.
Per la domanda la soluzione è analoga ma con un pozzo fittizio.

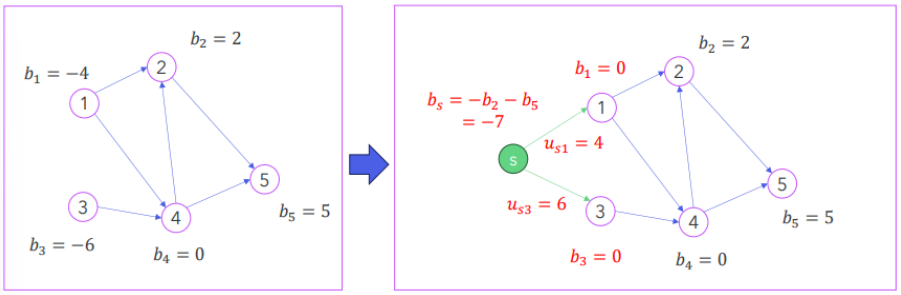


Figure 12: Esempio di trasformazione: eccesso di domanda

14.3.5 Ispezioni su una linea di produzione (esempio)

Descrizione: una linea di produzione ha n celle di lavorazione. Ogni lotto ha B pezzi che passano in sequenza nelle celle.
In ciascuna cella i un pezzo può diventare **difettoso** con probabilità p_i.
Lotti con difetti non possono essere inviati ai clienti.
Alla fine di ogni lavorazione può avvenire un'ispezione, se vengono trovati pezzi difettosi vengono scartati.
Un ispezione finale è obbligatoria, ma le ispezioni intermedie possono ridurre costi.
L'obiettivo è evitare lavorazioni superflue su pezzi destinati a essere scartati.
Ecco i costi:

- **Costo unitario di lavorazione** alla cella i: q_i
- **Costo fisso di ispezione** di un lotto all'uscita della cella j, supponendo che la precedente ispezione sia stata effettuata ad una cella $i < j$: f_{ij}
- **Costo unitario** per pezzo ispezionato: h_{ij}

Modellazione: partiamo col rappresentare il numero di pezzi non difettosi alla fine della lavorazione i:

$$B_i = B \cdot \prod_{k=1}^{i} (1 - p_k)$$

Costo totale (produzione e ispezione) alla cella $j > i$:

$$c_{ij} = f_{ij} + B_i \cdot h_{ij} + B_i \cdot \sum_{k=i+1}^{j} q_k$$

Per trasformare il problema, facciamo un passo alla volta; introduciamo una **cella fittizia**, la cella 0.
Poniamo il grafo $G = (N, A)$ con:

$$N = \{0, 1, 2, ..., n\}$$

$$A = \{(i, j) : i \in N \setminus \{n\}, j > i\}$$

Ad ogni arco è associato un costo c_{ij}, ovvero i costi totali calcolati prima.
La soluzione ottimale è quindi il cammino minimo dalla sorgente fittizia al nodo n.

15 Cammini di costo minimo

Il problema del cammino di costo minimo è uno dei più semplici ma più importanti su reti.

15.1 Definizione del problema generico

Definiamo un grafo **orientato** e **pesato** $G = (N, A)$:

- $N = \{1, ..., n\}$
- Ogni arco ha un costo $c_{ij} \in \mathbb{R}$

Cammino P: sequenza di archi che connette due nodi.
Costo di un cammino:

$$C(P) = \sum_{(i,j)\in P}^{c_{ij}}$$

Dati r (origine) e t (destinazione), P_{rt} è l'insieme dei cammini da r a t.
Il problema del cammino minimo consiste nel trovare lo **shortest path** fra r e t:

$$(SP) min C(P) : P \in P_{rt}$$

15.2 Particolarità e proprietà

La soluzione ottima può essere non unica, per questo dobbiamo includere un vincolo di interezza:

$$x \in \{0, 1\}^m$$

Inoltre, la soluzione potrebbe non esistere in quanto non esiste un cammino da r a t.
Per controllare l'inesistenza, si usa **BFS** o **DFS**, o addirittura **Dijkstra**.

Se esiste invece un ciclo orientato di costo **negativo** in modo che:

- Percorrendo il ciclo più volte, il costo totale diminuisce indefinitivamente
- Il problema è inferiormente limitato

Bisogna verificare che non esistano questi cicli, si può fare in tempo polinomiale.
Se invece un grafo NON ha questi cicli negativi, allora la soluzione ottimale è un **cammino semplice**.
Infatti, il problema del cammino di costo minimo coincide con il problema del cammino semplice di costo minimo.
Bisogna ovviamente accertarsi che il ciclo negativo faccia parte del cammino tra r e t, altrimenti non ha senso considerarlo per rendere il problema non semplice.
Esiste una condizione **sufficiente e non necessaria** per non avere cicli negativi: basta che tutti i costi siano **non negativi**.

Se abbiamo un cammino minimo tra due nodi, allora significa che abbiamo trovato anche il cammino minimo di ogni sotto-cammino.

15.3 Shortest Path Tree (SPT)

Si vuole trovare il cammino minimo di ogni cammino dalla sorgente a ogni nodo.
Si realizza un **albero** radicato nella sorgente e con $n-1$ archi (uno per nodo raggiungibile).
In questo modo bastano solo $n-1$ informazioni per descrivere tutti i cammini minimi da un nodo.
Considerando come sorgente r e ogni altro nodo come i allora possiamo formulare il problema cosi:

$$min(\sum_{i \neq r} C(P_i) : P_i \in P_{ri}, i \neq r)$$

Il problema può essere formulato anche come un Minimum Cost Flow:

$$b_i = \begin{cases} -(n-1), i = r \\ 1 \end{cases}$$

La radice è la sorgente di $n-1$ unità di flusso.
Ciascun nodo $i \neq r$ è un pozzo con domanda 1.
Ogni arco può essere visitato più volte se compare in più cammini, x_{ij} rappresenta quante volte.

Se il grafo a un nodo non raggiungibile dalla sorgente, si aggiunge un arco fittizio:

$$(r, i) \text{ con costo M} = (n-1)c_{max} + 1$$

15.3.1 Albero ottimo

Vogliamo verificare che un albero $T = (N, A_T)$ sia **ottimo** per il problema SPT.

- T è un SPT
- T è ottimo se non esiste un cammino da r a i con costo inferiore a $C(P_i^T)$

Definiamo un vettore di **etichette** d per cui:

$$d_i = C(P_i^T)$$

$$d_r = 0$$

Le etichette si possono calcolare, se per esempio abbiamo un arco $(i, j) \in A_T$ allora:

$$d_i + c_{ij} = d_j$$

Per ogni arco controlliamo se:

$$d_i + c_{ij} < d_j$$

- Se **si**, esiste un cammino migliore di P_j^T e quindi T non è ottimo.
- Se **no**, T è ottimo.

15.3.2 Teorema di ottimalità

Se un albero soddisfa queste condizioni (condizioni di **Bellman**):

$$\begin{cases} d_i + c_{ij} \geq d_j, \forall (i, j) \in A \\ d_r = 0 \end{cases}$$

Allora $T = (N, A_T)$ è un SPT radicato in r.